

A Lean 4 Formalization of Euclidean Domain Algorithms from a 1986 Icon Experimentation Package

Lars Warren Ericson¹

¹Catskills Research Company
¹lars.ericson@catskillsresearch.com

June 14, 2026

ORCID: 0000-0001-8299-9361

Primary Category: cs.LO (Logic in Computer Science)

Secondary Category: cs.SC (Symbolic Computation)

Original report: <https://zenodo.org/records/20561267>

Lean 4 formalization: <https://github.com/catskillsresearch/icon2lean>

Abstract

We describe a Lean 4 formalization of the algorithms and domain types from NYU Computer Science Technical Report #232, *An ICON Package for Experimenting with Euclidean Domains* (Ericson, 1986). The original system implemented Lipson’s catalog of procedures over integers, rationals, modular rings, polynomial rings, and truncated power series via a custom runtime dispatch mechanism in Icon. The present work separates three concerns: mathematical definitions grounded in Mathlib’s `EuclideanDomain` hierarchy, computable mirrors suitable for evaluation and regression testing, and report-formatting infrastructure that reproduces the 1986 benchmark output line-for-line. All fourteen application algorithms from Section 3 of the report are defined and typecheck without `sorry`; those grounded in Mathlib—chiefly integer gcd and extended Euclid—additionally carry machine-checked proofs. We classify each procedure by its epistemic status relative to Mathlib, enumerate the coherence obligations between the proof and computable layers, and state precisely what is theorem-backed versus regression-trusted. The formalization makes explicit the verification boundary that the 1986 package crossed only informally.

1 Introduction

In August 1986, at New York University’s Courant Institute of Mathematical Sciences, Lars Warren Ericson authored Technical Report #232 with the aim of implementing algebraic algorithms over multiple mathematical structures—integers, quotient rings, polynomials, and power series—following John Lipson’s *Elements of Algebra and Algebraic Computing*. Icon was chosen as the implementation language for its symbolic computation idioms. Lacking native type-classes, parameter inheritance, or object-oriented dispatch, the 1986 package realized generic division and arithmetic across distinct domains through a custom runtime dispatch system based on string reflection on procedure names (e.g., invoking `proc("div_" || type(a), 2)`).

Forty years later, we present a Lean 4 port of the report’s domain types (Section 2) and application algorithms (Section 3). The port is complete in the sense that every algorithm listed in the report’s application suite is defined and typecheck; no proof obligation is deferred via `sorry`. Source Icon listings and an OCR’d copy of the report are preserved alongside

the formalization at <https://github.com/catskillsresearch/icon2lean>; the original 1986 technical report is archived at <https://zenodo.org/records/20561267>.

The central methodological challenge of this port is not merely translation but *stratification*. Mathlib provides canonical mathematical objects—`Polynomial R`, `PowerSeries R`, and `EuclideanDomain.gcd`—but many key definitions are marked `noncomputable` in Lean. This non-computability arises because Mathlib constructs rely on classical axioms (such as the law of the excluded middle or classical choice) to define properties like polynomial degree or division, meaning they cannot be reduced by `#eval` or `native_decide`. The 1986 package, by contrast, was an *experimental, executable* system whose correctness was validated by evaluating and printing tables of results for selected inputs.

To bridge this paradigm gap, we maintain three layers—proof, computable, and report—and document explicitly which claims belong to each epistemic tier. We also translate Icon’s dynamic, mutable list paradigms into Lean’s pure, immutable, and tail-recursive structures without compromising the numerical correctness of the original suite.

Notation. Icon listings in the original report use *fancy notation* (§1.3 of the report): $\div$ denotes domain division, $\oplus$ addition, $\otimes$ multiplication; $F(\text{args}) \Leftarrow \text{body} \blacksquare$ is a procedure definition, where $\blacksquare$ terminates the definition, $\uparrow x$ returns x , and $\perp$ signals failure.

2 Domain Types

The report’s **quotient Euclidean domain** corresponds to Mathlib’s `EuclideanDomain`. Primitive domains are $\mathbb{Z}$ and $\mathbb{Q}$. Three constructors build composite domains, summarized in Table 1:

Table 1: Domain Constructors and Types

Constructor	Report Notation	Lean Type
Modular quotient $D/(e)$	<code>modulo(item, modulus)</code>	<code>ModularDomain R I / ModularInt n</code> ($\cong \mathbb{ZMod} \ n$)
Polynomial ring $D[x]$	<code>poly(terms)</code>	<code>PolyDomain R</code> ($\cong \text{Polynomial } R$)
Truncated series $T(D[[x]])_n$	<code>tpower(poly, N)</code>	<code>TruncPowerSeries R n</code> ($\cong R[x]/(X^n)$)

Formal power series $D[[x]]$ use `PowerSeries R` (for Newton power-series inversion); they are not Euclidean domains and carry no `EuclideanDomain` instance.

2.1 Primitive and Composite Instances

- **Integers:** `Int` / notation $\mathbb{Z}$. Instance: `EuclideanDomain Int`.
- **Rationals:** `Rat` / notation $\mathbb{Q}$. Field `Rat` induces `EuclideanDomain Rat`.
- **Modular rings:** General quotients via `Ideal.Quotient.mk`; units via `ZMod.unitOfCoprime`. There is no `LinearOrder` on `ZMod n`, matching the report’s `<0_modulo` predicate, which is always false (represented by $\perp$ in Icon).
- **Polynomials:** `Polynomial.degree` : `WithBot Nat`, with $\perp$ corresponding to the report’s “- infinity” for the zero polynomial. When F is a field, `Polynomial F` is a `EuclideanDomain`.
- **Truncated series:** `truncatePoly n p` implements multiply-then-truncate in $R[x]/(X^n)$.

In Lean 4, the truncated power series is constructed via quotients of the polynomial ring by the ideal generated by X^n :

```

noncomputable def truncIdeal (R : Type*) [CommRing R] (n : Nat) : Ideal (Polynomial R)
  :=
  Ideal.span {(X : Polynomial R) ^ n}

noncomputable abbrev TruncPowerSeries (R : Type*) [CommRing R] (n : Nat) :=
  Polynomial R / truncIdeal R n

```

2.2 Variable-base Digit Arithmetic (base.B)

Section 2.2 of the report defines variable-base digit arithmetic. No corresponding Mathlib infrastructure exists. For benchmark reproduction, we provide `BaseB.lean`, which implements digit vectors, conversion via `toNat` and `ofNat`, and arithmetic (`add`, `sub`, `mul`, `div`) via conversion through `Nat`. This layer functions as a **computable report helper** rather than a verified arbitrary-precision base- B ring, allowing us to accurately replicate digit formatting (including width padding and zero normalization) without incurring the proof overhead of a low-level digit-vector library.

3 Application Algorithms

The fourteen algorithms catalogued in Section 1.2 of the report and developed in Section 3 are fully realized in this formalization. Proof-layer definitions are generic over `EuclideanDomain` or `Polynomial R` where possible; the computable layer reimplements Icon control flow for $\mathbb{Q}[x]$, $\mathbb{F}_p[x]$, and truncated series.

3.1 Extended GCD and Modular Inverse

GCD. The Icon implementation is a recursive descent—`GCD(b, mod(a, b))` until $b = 0$ —following the standard Euclidean algorithm. In the proof layer, `euclideanGcd` is the gcd and `euclid` the extended gcd, both generic over any `EuclideanDomain`. On $\mathbb{Z}$, `euclidInt` and `euclidZ` delegate to Mathlib’s `Int.gcdA` and `Int.gcdB`. The computable layer provides `CompPoly.gcd` and `ModPoly.gcd`, which follow the Icon state machine and are guarded by a fuel parameter to satisfy Lean’s termination checker.

EUCLID. The Icon extended gcd maintains a triple (d, s, t) satisfying the Bézout identity $d = s \cdot A + t \cdot B$ throughout the loop. The canonical report example is `EUCLID(84, 54) = (6, 2, -3)`, meaning $\text{gcd}(84, 54) = 6 = 2 \cdot 84 + (-3) \cdot 54$.

INVERSE. Given modulus m and element a , Icon computes `gst := EUCLID(m, a)` and returns `mod(div(gst[3], gst[1]), m)` when `unit(gst[1])`—that is, when the gcd is a unit, so that $t \cdot a \equiv 1 \pmod{m}$. The argument order `EUCLID(m, a)` (modulus first) is a fixed part of the interface contract and is preserved in the Lean port. The proof layer provides `modularInverse` on $\mathbb{Z}$; `CompPoly.inverse` and `ModPoly.inverse` cover polynomial domains.

Lean 4 Certificate

```

Euclidean.lean | theorem euclid_bezout : Bezout identity for generic Euclidean
domains

theorem euclid_bezout {α : Type*} [EuclideanDomain α] [DecidableEq α] (a b : α)
  :
  euclideanGcd a b = a * EuclideanDomain.gcdA a b + b * EuclideanDomain.gcdB a
  b

```

Lean 4 Certificate

```
Euclidean.lean | theorem euclidZ_bezout : Bezout's identity for infinite-
precision integers

theorem euclidZ_bezout (A B : ℤ) :
  (euclidZ A B).1 = A * (euclidZ A B).2.1 + B * (euclidZ A B).2.2
```

3.2 Chinese Remainder and Diophantine Equations

CRA1, CRA2, CRA. Icon's **CRA1** is the recursive two-argument lifting step based on Niven and Zuckerman's linear congruence reduction; **CRA2** is the pairwise combination built from **INVERSE** and **mod**; **CRA** is the list driver. The Lean implementation resides in **Congruence.lean** and uses **iconMod** (Lean's **Int.emod**) to match Icon's non-negative remainder convention. Report examples include scalar CRA tables and a polynomial CRA producing $u(x) = 183 + 238x$.

DIOPHANTINE. Uses **EUCLID** to find the gcd, then **CRA1** to lift to a particular solution, with a branch on $|b| < |a|$. Lean implementation is provided in **Diophantine.lean**.

3.3 Polynomial Remainder Sequences

MOD_RS. The Euclidean remainder sequence: $\text{MOD_RS}(a, b) = [a] ++ \text{MOD_RS}(b, \text{mod}(a, b))$. The proof layer provides **Polynomial.modRS**; the computable layer, **CompPoly.modRS**. The report exhibit is a six-term $\mathbb{Q}[x]$ sequence ending in zero.

PREM. The pseudo-remainder: scale the dividend by $\text{lc}(q)^{\deg(p) - \deg(q) + 1}$, then take the ordinary remainder. Lean: **prem** in both layers.

E_PRS and S_PRS. **E_PRS** is the Euclidean PRS using **PREM** in place of ordinary **mod**; **S_PRS** is the Collins–Brown subresultant PRS. Both are provided in proof and computable layers. These algorithms are the most algebraically demanding in the suite: Mathlib provides polynomial **%** and **EuclideanDomain.gcd** over fields, but not pseudo-remainder, intermediate-coefficient-swell remainder sequences, or Collins–Brown subresultant PRS as named definitions matching Lipson's formulations.

3.4 Interpolation, FFT, and Power-Series Inversion

NIA. Newton interpolation on a list of points. Proof layer: **newtonInterpolation**; computable layer: **CompPoly.nia**.

FFT and FFI. **FFT** follows Cooley–Tukey decimation on even- and odd-indexed coefficient splits. **FFI** polynomializes the input, calls **FFT** with ω^{-1} as the primitive root, and scales by $1/N$. Proof layer: **evenTerms**, **oddTerms**, **fftCoeffs**, **ffi** in **Fft.lean**; computable twins operate on **List CRat**.

NPSI. Newton iteration for truncated power-series inversion. Proof layer: **npsi**, **npsiTrunc**; computable layer: **CompTPS.npsi**.

4 Architecture

The 1986 package conflated three concerns: representing Euclidean domains, executing algorithms, and printing benchmark tables in a fixed format. The Lean port separates them deliberately.

4.1 Three Layers

The formalization is structured into three layers to balance mathematical rigor, executability, and faithful stdout generation, as mapped in Table 2:

Table 2: The Three-Layer Architecture

Layer	Role	Representative Modules
Proof / Canonical	Mathlib-backed types and <code>noncomputable</code> definitions	<code>Types.lean</code> , <code>Domains.lean</code> , <code>Euclidean.lean</code>
Computable / Eval	Computable mirrors for kernel-level evaluation	<code>ComputablePoly.lean</code> , <code>ComputableTPS.lean</code>
Report / Print	Icon-style formatters and benchmark drivers	<code>Print.lean</code> , <code>Report.lean</code>

Design Rationale. Mathlib’s polynomial and gcd infrastructure provides the correct mathematical substrate, but many key definitions—`EuclideanDomain.gcd` on `Polynomial F` in particular—are marked `noncomputable` in Lean because they depend on classical axioms such as `Classical.choice`. The kernel cannot reduce `noncomputable` terms, so `#eval` and `native_decide` cannot execute them directly. We therefore maintain computable mirrors (`CompPoly`, `CRat`, `CompTPS`) together with a boundary map `CompPoly.toMathlib`. Coherence lemmas connecting these two layers are stated but not yet proved. Integer and `ZMod p` arithmetic requires no mirror: kernel `Int` and `decide` suffice.

Matching Icon Semantics. The computable layer follows Icon’s algorithms literally—not Mathlib’s canonical division—because the validation criterion is exact reproduction of the 1986 benchmark output. This entailed several non-obvious implementation choices:

- **div_poly / mod_poly:** Each quotient step accumulates a single term `qterm`, then subtracts `qterm * b` from the current remainder, rather than maintaining a running quotient polynomial.
- **EUCLID:** The extended gcd state is the tuple $(a_1, a_2, s_1, s_2, t_1, t_2)$ with initial values $(A, B, 1, 0, 0, 1)$; `INVERSE` calls `EUCLID(b, a)` with the modulus as the first argument.
- **MOD_RS:** Recursive remainder sequence; early exits in `div` for constant divisors affect intermediate term shapes.
- **S_PRS:** `subReduce` short-circuits when `prem` returns zero, matching Icon behavior.
- **base_B:** Digit lists accumulate LSB-first and are then reversed to MSB-first order.
- **Printing:** `Print.lean` reimplements Icon’s `print_*` family so stdout is diffable against the 1986 output, rather than relying on Lean’s `Repr` instances.

4.2 Module Correspondence

The physical layout of the Lean project corresponds directly to sections of the 1986 report (Table 3):

Table 3: Module Correspondence

Report Section	Proof Layer	Computable / Report Layer
§2 Domain Types	Types.lean, Domains.lean	—
§2 base_B	—	BaseB.lean
GCD, EUCLID, INVERSE	Euclidean.lean, Gcd.lean	ComputableAlg.lean, ModPoly.lean
CRA1, CRA2, CRA	Congruence.lean	same (kernel Int)
DIOPHANTINE	Diophantine.lean	same
MOD_RS, PREM, E_PRS, S_PRS	Polynomial.lean	ComputablePoly.lean, ComputableAlg.lean
NIA	Interpolation.lean	CompPoly.nia in ComputableAlg.lean
FFT, FFI	Fft.lean	CompPoly.fftCoeffs, CompPoly.ffi
NPSI	PowerSeries.lean	ComputableTPS.lean
Icon printing	—	Print.lean
Full benchmark	—	Report.lean

5 Verification and Trust

We classify each component by epistemic status. The 1986 report lists fourteen application procedures; the congruence stack is **CRA1** / **CRA2** / **CRA** (Chinese remainder).

5.1 Trust Tiers

The formalization comprises three trust tiers based on their verification status relative to Mathlib (Table 4):

Table 4: Trust Tiers

Tier	Meaning	Examples
Tier A	Delegates to proved Mathlib facts	euclid_bezout, euclidInt_bezout, Ring laws in Domains.lean
Tier B	Icon-faithful; matched via regression	CompPoly.div, modRS, ePRS, sPRS, fftCoeffs, BaseB
Tier C	Stated coherence obligations (unproved)	CompPoly.toMathlib homomorphism lemmas

Lean 4 — Future Formalisation

Computability.lean | lemma toMathlib_homomorphism: Homomorphism between computable and canonical models

```
lemma toMathlib_add (p q : CompPoly) : toMathlib (add p q) = toMathlib p +
  toMathlib q
lemma toMathlib_mul (p q : CompPoly) : toMathlib (mul p q) = toMathlib p *
  toMathlib q
```

The majority of the port sits in Tier B. The immediate goal was faithfully reproducing the 1986 experimental package; certifying every Lipson procedure against Mathlib’s canonical polynomial API is future work.

5.2 Procedure-by-Procedure Analysis

- **GCD / EUCLID:** On integers, Mathlib supplies `Int.gcd`, `Int.gcdA`, `Int.gcdB`, and Bézout’s lemma; this package adds thin wrappers shaped to the report’s interface and

printing conventions. On $\mathbb{Q}[x]$ and $\mathbb{F}_p[x]$, Mathlib provides `EuclideanDomain.gcd`, but `CompPoly.gcd` and `ModPoly.gcd` are separate Icon-style Euclidean loops with fuel bounds, validated against Icon output tables but not yet proved equal to `EuclideanDomain.gcd` via `toMathlib`. Fuel-guarded definitions may fail to terminate on inputs outside the report’s test corpus; on those inputs they produce the correct gcd.

- **INVERSE:** Modular-integer inverse reduces to Mathlib’s extended gcd. Polynomial inverse has no matching Mathlib API; the package adds a concrete, testable procedure matching the 1986 benchmarks.
- **CRA / DIOPHANTINE:** Mathlib contains Chinese remainder theorems in commutative algebra, but not the Lipson/Ericson recursive `CRA1` lifting step, the `CRA2` pairwise combiner, or the list-shaped `CRA` driver with Icon `mod` semantics. Correctness rests on classical number theory; the implementation is validated by `native_decide` on the report’s congruence tables rather than formalized as theorems.
- **PRS family:** The package adds pseudo-remainder, remainder sequences, `E_PRS`, and Collins–Brown `S_PRS`—standard computer-algebra algorithms from Lipson—implemented in both proof and computable layers. Neither gcd correctness of the last nonzero sub-resultant nor equivalence with `EuclideanDomain.gcd` is proved. `MOD_RS` is pedagogical, its output exhibiting coefficient swell; correctness rests on full report parity and targeted coefficient checks.
- **FFT / FFI:** Mathlib has no Cooley–Tukey API. There is no proof that `fftCoeffs` computes a DFT or that `ffi` recovers the unique interpolating polynomial. Correctness assumes a suitable primitive root of unity ω and that ω^{-1} exists—the same implicit preconditions as in the 1986 Icon code. Bugs in `INVERSE` propagate to `FFI`.
- **NIA / NPSI:** Newton interpolation and truncated power-series inversion are added as standalone definitions; Mathlib contains related infrastructure but not these exact procedures.
- **base_B / printing:** Entirely package-specific; infrastructure for report diffing, not mathematics.

The comparative coverage is detailed in Table 5:

Table 5: Comparison: Mathlib versus this formalization

Procedure	In Mathlib?	Contribution of this work
GCD / EUCLID on $\mathbb{Z}$	Yes	Report API, printing, tables
GCD / EUCLID on $\mathbb{Q}[x]$, $\mathbb{F}_p[x]$	Yes (abstract)	Icon-faithful computable gcd/euclid + report parity
INVERSE on $\mathbb{Z}$	Yes	Icon argument order, <code>Option</code> + error strings
INVERSE on polynomials	No	Full procedure + tests
CRA1 / CRA2 / CRA	Theory only	Exact recursive Icon algorithms + <code>iconMod</code>
DIOPHANTINE	No	Composition of gcd + <code>CRA1</code>
MOD_RS / PREM / E_PRS / S_PRS	No	Full PRS suite (proof + computable)
NIA	No	Newton interpolation
FFT / FFI	No	Cooley–Tukey + interpolation pipeline
NPSI	Partial	Newton truncated iteration as in report
base_B	No	Digit-vector arithmetic for §2 tables
Icon stdout	No	<code>Print.lean</code> , <code>Report.lean</code> , comparison tooling

6 Experimental Validation and Typo Resolution

The 1986 package validated algorithms by printing tables of results on selected inputs. We replicate this methodology at two granularities:

- **Unit-level checks:** `Tests.lean` uses `native_decide` on coefficients and congruences for the report’s §3.1–§3.2 examples.
- **End-to-end report parity:** Icon `tests.icn main()` and Lean `lake exe iconReport` produce identical 85-line stdout after normalizing timing metadata and a small number of EUCLID fraction parenthesizations. This integration test exercises all three layers—proof, computable, and printing—together.

The validation coverage is outlined in Table 6:

Table 6: Validation Matrix

Report Section	Example	Validation
§2	<code>base_B</code> add/mul/div tables	report diff
§3.1.1	EUCLID(84, 54) = (6, 2, −3)	<code>native_decide</code> + report
§3.1.2	INVERSE table ($\mathbb{Z}$, $\mathbb{Q}[x]$, $\mathbb{F}_2[x]$)	<code>native_decide</code> + report
§3.1.3	CRA1, CRA2, CRA (incl. $u(x) = 183 + 238x$)	<code>native_decide</code> + report
§3.1.4	Diophantine particular solutions	<code>native_decide</code> + report
§3.2.1	MOD_RS six-term sequence	<code>native_decide</code> + report
§3.2.2	PREM rows	<code>native_decide</code> + report
§3.2.3–§3.2.4	E_PRS, S_PRS	<code>native_decide</code> + report
§3.3	NIA, FFT, FFI, NPSI	<code>native_decide</code> + report

6.1 Reconciling the PREM Typo

A notable outcome of this formalization was the identification and resolution of a historical typo in Section 3.2.2 of the 1986 Technical Report. The printed table lists the pseudo-remainder (`prem`) for Row 1 as:

$$\begin{aligned} \text{PREM}(2042542724z + 17851334z \cdot X, 5851259279846738252460z) \\ = -5851259279846738252460000000000z \end{aligned}$$

However, when executing the original 1986 Icon code (`all.icn.txt`), Row 1 evaluates to `0zq`. This is mathematically correct: since the divisor q is a constant of degree 0, the degree difference $d = \deg(p) - \deg(q) = 1 - 0 = 1$. The algorithm scales the dividend by $\text{lc}(q)^{d+1} = \text{lc}(q)^2$, yielding:

$$\text{rem}(\text{lc}(q)^2 \cdot p(x), q) = \text{rem}(\text{lc}(q)^2 \cdot (2042542724 + 17851334X), \text{lc}(q))$$

Because q is a scalar constant ($\text{lc}(q)$), any polynomial divided by it yields a remainder of 0 over the field of fractions. The Lean formalization accurately matches this runtime reality, yielding `0zq` in both the computable layer and the final output report, thus resolving a forty-year-old documentation error through active execution.

7 Omissions

Per the report itself, we omit utilities that are not part of the mathematical core:

- **Runtime dispatch** — generic dispatch of operations such as `div` and `mod` by domain type is replaced by Lean typeclasses and fixed modules.
- **Timer fidelity** — `settime` / `showtime` in §3.4 produced meaningful timing data in Icon; Lean prints `[0 msecs]`, and the comparison tooling normalizes those lines.
- **Verified base- B long arithmetic** — `BaseB` reproduces benchmarks via `Nat`, not a proved digit ring.

8 Conclusion and Future Work

The 1986 Icon package validated algorithms on selected inputs by printing tables. The Lean port automates the same validation: unit tests via `native_decide` and a full line-by-line report diff across 85 lines. All fourteen Section 3 application algorithms are defined and typecheck without `sorry`. Section 2 domain types use Mathlib instances; `base_B` and the Icon printing conventions are reproduced for parity.

Provability Status. Integer gcd and extended Euclid are anchored in Mathlib theorems (Tier A). The polynomial PRS family, FFT/FFI, and the `CompPoly` computable layer are regression-trusted (Tier B), resting on the same epistemic footing as the 1986 Icon package. Coherence obligations connecting the proof and computable layers are identified (Tier C) but not yet discharged.

Future Work. The most important open items are the Tier C coherence proofs (`toMathlib` for `CompPoly` and `CompTPS`) and the semantic theorems currently at Tier B: that CRA satisfies its congruence system, that `S_PRS` yields the correct gcd, and that FFT and FFI compose into a correct interpolation identity. Lean makes each obligation precise and stateable; the present formalization provides canonical definitions and Icon-faithful executables side by side, without prematurely replacing the latter with Mathlib calls that would break historical parity.

References

- [1] L. W. Ericson, *An ICON Package for Experimenting with Euclidean Domains*, NYU Computer Science Technical Report #232, August 1986. <https://zenodo.org/records/20561267> (DOI: <https://doi.org/10.5281/zenodo.20561267>).
- [2] *icon2lean*: Lean 4 formalization of the 1986 Icon package. <https://github.com/catskillsresearch/icon2lean>.
- [3] J. Lipson, *Elements of Algebra and Algebraic Computing*, Benjamin/Cummings, 1981.
- [4] I. Niven, H. S. Zuckerman, and H. L. Montgomery, *An Introduction to the Theory of Numbers*, 5th ed., John Wiley & Sons, 1991.
- [5] L. de Moura and S. Ullrich, *The Lean 4 Theorem Prover and Programming Language*, CADE 2021.
- [6] The Mathlib Community, *The Lean mathematical library*, CPP 2020.